Comparación experimental de Shell Sort y Quick Sort

Experimental Comparison of Shell Sort and Quick Sort

Jhosselin Rouss Quenta Salluca
Escuela Profesional de Ingeniería Informática y Sistemas
Universidad Nacional Jorge Basadre Grohmann
Tacna, Perú
Correo institucional: *jrquentas@unjbg.edu.pe*

2026

Abstract

El objetivo de este trabajo fue comparar experimentalmente los algoritmos Shell Sort y Quick Sort mediante una implementación en C++. Se evaluaron entradas de 1 000, 10 000 y 100 000 elementos, con patrones aleatorios, ascendentes y descendentes. Se realizaron 30 mediciones por cada combinación de tamaño, patrón y algoritmo, para un total de 540 registros. Se analizaron los tiempos de ejecución, las comparaciones, los intercambios y la profundidad de recursión cuando correspondía. Los tiempos medios mostraron que Quick Sort fue más rápido en las nueve combinaciones evaluadas. Para entradas aleatorias de 100 000 elementos, su tiempo medio fue de 22.733 ms, frente a 80.405 ms de Shell Sort. Estos resultados corresponden a las implementaciones y condiciones utilizadas, por lo que no demuestran que Quick Sort sea siempre superior. Se concluye que el tamaño y el patrón de entrada deben considerarse al comparar algoritmos de ordenamiento.

Palabras clave: algoritmos de ordenamiento, Shell Sort, Quick Sort, C++, análisis experimental.

Abstract

This study experimentally compared Shell Sort and Quick Sort using a C++ implementation. Input sizes of 1,000, 10,000, and 100,000 elements were evaluated with random, ascending, and descending patterns. Thirty measurements were performed for each combination of input size, pattern, and algorithm, producing 540 records. Execution times, comparisons, swaps, and recursion depth when applicable were analyzed. Quick Sort had a lower mean execution time in all nine tested conditions. For random inputs of 100,000 elements, its mean time was 22.733 ms, compared with 80.405 ms for Shell Sort. These findings apply to the implementations and experimental conditions used and do not establish that Quick Sort is always faster. Input size and pattern should be considered when comparing sorting algorithms.

Keywords: sorting algorithms, Shell Sort, Quick Sort, C++, experimental analysis.

1 Introducción

Los algoritmos de ordenamiento permiten organizar datos de acuerdo con un criterio, como el orden ascendente. Su eficiencia resulta importante cuando aumenta la cantidad de elementos que deben procesarse.

Shell Sort mejora el ordenamiento por inserción al comparar elementos separados por intervalos que disminuyen progresivamente. Quick Sort selecciona un pivote y divide los elementos en partes que se ordenan mediante llamadas recursivas.

La eficiencia de un algoritmo depende del tamaño de entrada, la distribución de los datos, la implementación y el entorno de ejecución. Por ello, resulta conveniente combinar el análisis teórico con mediciones experimentales (Cormen et al., 2022; Hoare, 1962; Shell, 1959).

1.1 Objetivo

Comparar experimentalmente Shell Sort y Quick Sort considerando el tiempo de ejecución, las comparaciones, los intercambios y la profundidad de recursión cuando corresponda.

1.2 Hipótesis descriptiva

Se espera observar diferencias de rendimiento entre ambos algoritmos según el tamaño y el patrón de entrada. Esta expectativa se evalúa mediante los datos obtenidos, sin asumir de antemano que uno será siempre más rápido.

2 Marco teórico

2.1 Shell Sort

Shell Sort ordena los elementos utilizando intervalos que se reducen progresivamente hasta llegar a uno. En esta última etapa se realiza un ordenamiento por inserción sobre el arreglo.

Su complejidad depende de la secuencia de intervalos utilizada. En esta implementación, el intervalo se divide entre dos en cada etapa. Su peor caso puede ser cuadrático.

2.2 Quick Sort

Quick Sort selecciona un pivote y particiona los elementos según su valor. Después ordena recursivamente las partes resultantes.

Su complejidad promedio es $O(n \log n)$, mientras que su peor caso es $O(n^2)$. Su rendimiento depende de las particiones generadas. En este proyecto se utiliza un pivote central.

2.3 Comparación teórica

Cuadro 1: Características teóricas de los algoritmos

Algoritmo	Mejor caso	Caso promedio	Peor caso	Estabilidad
Shell Sort	Depende de los intervalos	Depende de los intervalos	Puede ser $O(n^2)$	Inestable
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	Inestable

Nota. La complejidad de Shell Sort depende de la secuencia de intervalos. Fuente: elaboración propia a partir de la literatura consultada.

La estabilidad indica si los elementos con claves iguales conservan su orden relativo original. Las implementaciones estudiadas no garantizan esta propiedad.

3 Metodología

3.1 Entorno de ejecución

El experimento se implementó en C++ y se compiló con G++ 15.2.0 en MSYS2 UCRT64 sobre Windows. Para medir el tiempo se utilizó `std::chrono::steady_clock`. El tiempo se expresó en milisegundos.

El modelo del procesador, la memoria RAM y las opciones exactas de compilación no se registraron en los datos disponibles. Por tanto, estos elementos constituyen información pendiente para una reproducción más completa del experimento.

3.2 Datos experimentales

Se utilizaron tres tamaños de entrada:

$$n \in \{1000, 10000, 100000\}.$$

Se evaluaron tres patrones: aleatorio, ascendente y descendente. Se realizaron 30 mediciones por cada combinación de tamaño, patrón y algoritmo. El total esperado fue:

$$3 \times 3 \times 30 \times 2 = 540.$$

Para generar los datos aleatorios se utilizó la semilla 12345. En cada repetición, ambos algoritmos recibieron una copia de la misma entrada base. Así se mantuvieron iguales los datos comparados bajo cada condición.

Las repeticiones corresponden a mediciones de una misma entrada base por condición, no a 30 conjuntos independientes. La semilla fija permite reproducir la entrada aleatoria, pero no representa todas las distribuciones posibles.

3.3 Métricas y procesamiento

Los resultados se almacenaron en `results/comparacion.csv`. Se registraron el algoritmo, el tamaño, el patrón, la repetición, el tiempo, las comparaciones, los intercambios, la profundidad de recursión, la semilla, el identificador del commit y la comprobación del ordenamiento.

Se verificó que el archivo contuviera 540 registros y que ninguno presentara un resultado sin ordenar. Posteriormente, se calcularon los tiempos medios para cada algoritmo, tamaño y patrón. Las estadísticas globales que mezclan condiciones diferentes no se utilizaron para establecer cuál algoritmo fue más rápido.

4 Resultados

4.1 Tiempos medios de ejecución

La Tabla 2 presenta los tiempos medios de las 30 mediciones realizadas para cada combinación experimental.

Cuadro 2: Tiempo medio de ejecución por algoritmo y condición

n	Patrón	Shell Sort (ms)	Quick Sort (ms)
1 000	Aleatorio	0.524	0.249
1 000	Ascendente	0.165	0.097
1 000	Descendente	0.307	0.106
10 000	Aleatorio	8.862	3.332
10 000	Ascendente	2.376	1.231
10 000	Descendente	4.703	1.335
100 000	Aleatorio	80.405	22.733
100 000	Ascendente	16.907	8.408
100 000	Descendente	31.296	8.898

Nota. Los tiempos se expresan en milisegundos. Fuente: elaboración propia a partir de los datos experimentales.

Quick Sort presentó un menor tiempo medio en las nueve combinaciones evaluadas. Para 100 000 elementos aleatorios, Shell Sort registró 80.405 ms, mientras que Quick Sort registró 22.733 ms.

4.2 Comparación relativa

Para facilitar la interpretación, se calculó la razón entre el tiempo medio de Shell Sort y el de Quick Sort. Una razón mayor que uno indica que Shell Sort tardó más en esa condición.

Cuadro 3: Razón entre los tiempos medios de Shell Sort y Quick Sort

n	Patrón	Razón Shell/Quick
1 000	Aleatorio	2.11
1 000	Ascendente	1.71
1 000	Descendente	2.90
10 000	Aleatorio	2.66
10 000	Ascendente	1.93
10 000	Descendente	3.52
100 000	Aleatorio	3.54
100 000	Ascendente	2.01
100 000	Descendente	3.52

Fuente: cálculo propio a partir de los tiempos medios.

La mayor razón se observó con 100 000 elementos aleatorios, donde el tiempo medio de Shell Sort fue aproximadamente 3.54 veces el de Quick Sort. Esta razón describe los tiempos de las implementaciones evaluadas y no constituye una propiedad universal de los algoritmos.

4.3 Gráfico de tiempos para entradas aleatorias

La Figura 1 compara los tiempos medios medidos para entradas aleatorias. Se utilizan escalas logarítmicas para facilitar la lectura de los diferentes tamaños y tiempos.

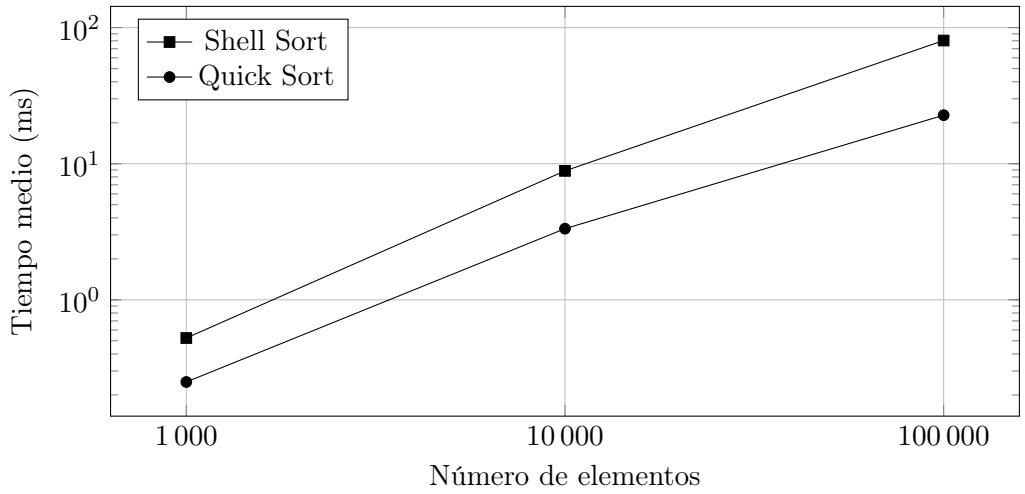


Figura 1: Tiempos medios para entradas aleatorias.

Fuente: elaboración propia a partir de los promedios experimentales.

4.4 Comparación para 100 000 elementos

La Figura 2 muestra los tiempos medios para los tres patrones cuando el tamaño de entrada fue de 100 000 elementos.

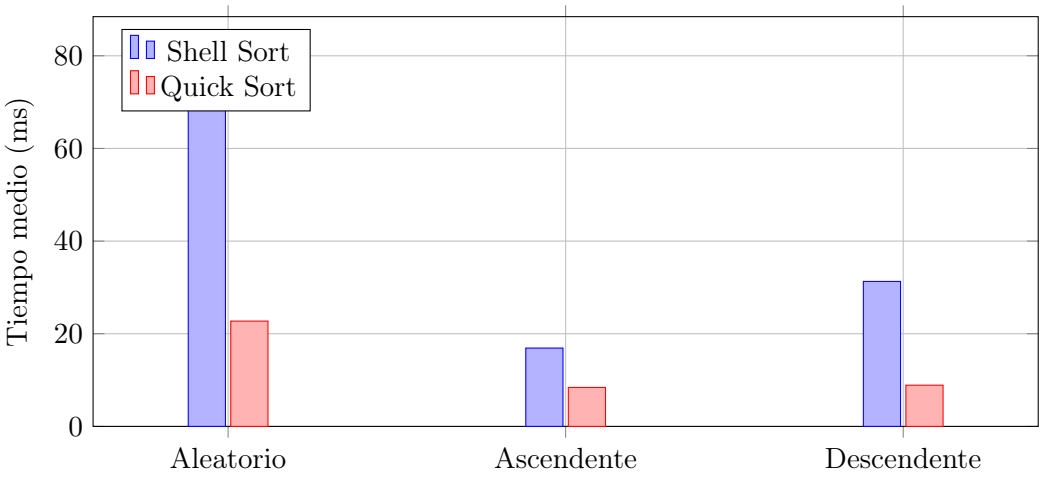


Figura 2: Tiempos medios según el patrón para 100 000 elementos.

Fuente: elaboración propia a partir de los promedios experimentales.

En ambos algoritmos, el patrón de entrada estuvo asociado con diferencias en el tiempo de ejecución. Para este tamaño, Quick Sort presentó tiempos medios menores en los tres patrones.

4.5 Verificación del ordenamiento

Los 540 registros del archivo CSV indicaron que las salidas estaban ordenadas. Esta verificación confirma el resultado de las ejecuciones registradas, aunque no sustituye otras pruebas de corrección para entradas diferentes.

5 Discusión

Los tiempos medios obtenidos favorecieron a Quick Sort en las nueve combinaciones evaluadas. La diferencia fue especialmente notoria para las entradas aleatorias de mayor tamaño. Estos resultados son compatibles con el buen rendimiento promedio esperado de Quick Sort, pero no permiten afirmar que siempre será más rápido que Shell Sort.

Shell Sort depende de la secuencia de intervalos utilizada, mientras que Quick Sort puede presentar un rendimiento desfavorable cuando las particiones son poco equilibradas. Por ello, los resultados experimentales deben interpretarse junto con las complejidades teóricas de ambos algoritmos.

Las 30 mediciones de cada combinación utilizaron la misma entrada base. Esto permite comparar los tiempos bajo condiciones controladas, pero no representa 30 conjuntos de datos independientes. Además, el uso de una sola semilla limita la variedad de entradas aleatorias consideradas.

El sistema operativo, el hardware y otros procesos activos también pueden influir en los tiempos medidos. Por último, aunque los 540 registros superaron la comprobación de ordenamiento, el análisis de comparaciones, intercambios y profundidad de recursión permitiría ampliar la evaluación.

6 Conclusiones

1. Se implementaron y compararon Shell Sort y Quick Sort en C++, utilizando tres tamaños y tres patrones de entrada.
2. Se obtuvieron 540 registros correspondientes a 30 mediciones por cada combinación de algoritmo, tamaño y patrón.
3. Quick Sort presentó menores tiempos medios en las nueve condiciones evaluadas. Para 100 000 elementos aleatorios, su tiempo medio fue de 22.733 ms, frente a 80.405 ms de Shell Sort.
4. Todos los registros superaron la comprobación de ordenamiento. Sin embargo, el tiempo no es el único indicador para evaluar un algoritmo.
5. Las conclusiones se limitan a las implementaciones y condiciones utilizadas. Se recomienda ampliar las pruebas con diferentes semillas y analizar la variabilidad de los tiempos obtenidos.

Disponibilidad de los datos

Los datos experimentales se encuentran en el archivo `results/comparacion.csv` del repositorio del proyecto. El código fuente permite revisar la implementación de los algoritmos y el procedimiento de medición. Los promedios presentados en este artículo se calcularon a partir de esos registros.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Hoare, C. A. R. (1962). Quicksort. *The Computer Journal, 5*(1), 10–16. `https://doi.org/10.1093/comjnl/5.1.10`

Shell, D. L. (1959). A high-speed sorting procedure. *Communications of the ACM, 2*(7), 30–32. `https://doi.org/10.1145/368370.368387`